



CortexAI — Everything Else You're Missing

This document covers every backend topic that does NOT have its own dedicated readme yet. It fills the gaps between your existing documentation files.

Table of Contents

1. [Chat Service Deep-Dive](#)
 2. [Agent Service Deep-Dive](#)
 3. [All Database Schemas & Relationships](#)
 4. [Complete API Endpoints Reference](#)
 5. [How to Start the Entire Project](#)
 6. [The `shared/` Folder Pattern](#)
 7. [Docker Compose for Redis](#)
 8. [The `.env` Files — What Each Service Needs](#)
 9. [Package.json Scripts & Dependencies Explained](#)
 10. [Inter-Service HTTP Calls \(Agent → Chat\)](#)
 11. [Error Scenarios & What Breaks](#)
 12. [Unanswered "Why" Questions](#)
-

1. Chat Service Deep-Dive

Location: `backend/services/CHAT/`

Port: `8002`

Database: MongoDB Atlas → `chat` database

Purpose: Owns ALL conversation and message data. No other service directly touches this data.

Entry Point: `index.js`

```
import express from "express"
import dotenv from "dotenv"
import connectDb from "../config/db.js"
import router from "../routes/chat.routes.js"
dotenv.config()

const port = process.env.PORT || 8002
const app = express()
app.use(express.json()) // Parses JSON bodies – needed for saveMessage, updateConversation
app.use("/", router) // All routes mounted at root
app.listen(port, () => { connectDb() })
```

Key difference from Gateway: The Chat Service uses `express.json()` because it needs to parse request bodies (POST data). The Gateway does NOT use `express.json()` because it proxies raw requests without parsing them.

Controller: `chat.controller.js` – 5 functions

`createConversation`

```
export const createConversation = async (req, res) => {
  const userId = req.headers["x-user-id"] // From proxyWithHeader
  const conversation = await Conversation.create({ userId })
  return res.status(200).json(conversation)
}
```

- **Who calls it:** Frontend sidebar "New Chat" button → `GET /chat/create-conversation` → Gateway proxies to Chat
- **What it creates:** A new Conversation document with `title: "New Chat"` (default) and `userId` from header
- **Returns:** The full conversation object including `_id`, `title`, `userId`, `createdAt`, `updatedAt`
- ⚠️ **Design note:** Uses GET method but creates a resource — should ideally be POST

getConversations

```
export const getConversations = async (req, res) => {
  const userId = req.headers["x-user-id"]
  const conversations = await Conversation.find({ userId }).sort({ updatedAt: -1 })
  return res.status(200).json(conversations)
}
```

- **Who calls it:** Frontend sidebar on load → GET /chat/get-conversation
- **What it returns:** Array of all conversations belonging to this user, sorted newest-updated first
- **Why updatedAt: -1:** -1 = descending order. When a user sends a message, the conversation's updatedAt changes, pushing it to the top of the sidebar

updateConversation

```
export const updateConversation = async (req, res) => {
  const { id, title } = req.body
  const conversation = await Conversation.findByIdAndUpdate(id, { title }, { new: true })
  return res.status(200).json(conversation)
}
```

- **Who calls it:** Not currently called from frontend (feature not implemented in UI yet)
- **Purpose:** Rename a conversation (e.g., from "New Chat" to "Redis Discussion")
- **{ new: true }:** Returns the document AFTER the update, not before

saveMessage

```
export const saveMessage = async (req, res) => {
  const { conversationId, role, content } = req.body
  const message = await Message.create({ conversationId, content, role })
  return res.status(200).json(message)
}
```

- **Who calls it TWO different ways:**
 - i. **Agent Service** (server-to-server):
axios.post("http://localhost:8002/save-message", {...})

→ Saves user prompt AND AI response (2 separate calls)

ii. **Frontend** could also call it through Gateway: `POST /chat/save-message`

- **Role field:** Either `"user"` or `"assistant"` — enforced by Mongoose `enum`
- **conversationId:** Links the message to its parent Conversation document

getMessage

```
export const getMessage = async (req, res) => {
  const messages = await Message.find({ conversationId: req.params.conversationId })
    .sort({ createdAt: -1 })
  return res.status(200).json(messages)
}
```

- **Who calls it:** Frontend ChatArea component when user clicks a conversation
- **Route:** `GET /get-message/:conversationId` (`:conversationId` = URL parameter)
- **⚠️ Note:** Sorted `createdAt: -1` (newest first) — the frontend might need to reverse this for display

Routes: `chat.routes.js`

```
router.get("/create-conversation", createConversation) // Should be POST
router.get("/get-conversation", getConversations)
router.post("/save-message", saveMessage)
router.get("/get-message/:conversationId", getMessage)
router.post("/update-conversation", updateConversation)
```

Path prefix stripping: Remember, the Gateway does `app.use("/chat", proxy(...))`. So when the browser requests `GET /chat/get-conversation`, the Chat Service receives `GET /get-conversation` — the `/chat` prefix is stripped by the proxy.

2. Agent Service Deep-Dive

Location: `backend/services/agent/`

Port: `8003`

Database: MongoDB Atlas → `agent` database (connected but not currently used for storage)

Purpose: Receives user prompts, runs them through the LangGraph AI pipeline, and saves results via the Chat Service.

Entry Point: `index.js`

```
import "dotenv/config"           // Alternative to dotenv.config() – single import line
import express from "express"
import connectDb from "../config/db.js"
import router from "../routes/route.js"

const port = process.env.PORT || 8003
const app = express()
app.use(express.json())
app.use("/", router)
app.listen(port, () => { connectDb() })
```

Key differences:

- Uses `import "dotenv/config"` instead of `dotenv.config()` — both work identically
- Connects to its own `agent` database (currently unused — no models defined)
- Has its own `node_modules` with LangChain, LangGraph, axios

Controller: `agent.controller.js` — The Central Orchestrator

```
export const agent = async (req, res) => {
  const { prompt, conversationId } = req.body

  // Step 1: Save user's message to Chat Service
  await axios.post(`${process.env.CHAT_SERVICE}/save-message`, {
    conversationId, role: "user", content: prompt
  })

  // Step 2: Run LangGraph
  const result = await graph.invoke({ prompt, conversationId })
  const aiResponse = result.aiResponse

  // Step 3: Save AI response to Chat Service
  await axios.post(`${process.env.CHAT_SERVICE}/save-message`, {
    conversationId, role: "assistant", content: aiResponse
  })

  // Step 4: Send response back to frontend
  return res.status(200).json({ aiResponse })
}
```

Why Agent saves messages (not the frontend)?

- The Agent is the one that generates the AI response — it knows both the user prompt and the AI reply.
- This ensures messages are saved atomically — if the AI fails, no half-saved messages.
- The frontend doesn't need to make 3 separate API calls (save user msg, get AI response, save AI msg).

Why Agent calls Chat Service HTTP API instead of directly accessing Chat's MongoDB?

- **Database-per-service rule:** Only the Chat Service should read/write its own database.
- If Agent directly accessed Chat's MongoDB, you'd have **tight coupling** — any schema change in Chat would break Agent.
- HTTP API is the contract — as long as the API stays the same, both services can change

internally.

Route: `route.js`

```
router.post("/chat", agent)
```

Full path from browser: `POST /agent/chat`

- Gateway receives: `POST /agent/chat`
- Gateway strips `/agent`, forwards: `POST /chat` to Agent Service
- Agent Service matches: `POST /chat` → `agent` controller

Agent stubs — What they are and why they exist:

```
// search.agent.js, coding.agent.js, pdf.agent.js, ppt.agent.js, vision.agent.js
export const searchagent = async (params) => {
  // Empty – not implemented yet
}
```

- These are **placeholder functions** for future agent implementations.
 - The LangGraph `graph.js` already imports and wires them into the graph.
 - The `router.js` LLM already knows about them and can route to them.
 - When you implement them, you only need to fill in the function body — the wiring is done.
-

3. All Database Schemas & Relationships

Three MongoDB Databases on Same Atlas Cluster

```
MongoDB Atlas Cluster (cluster0.p2k1stc.mongodb.net)
├─ auth database
│   └─ users collection      ← User documents
├─ chat database
│   ├── conversations collection ← Conversation documents
│   └─ messages collection    ← Message documents
└─ agent database
    └─ (empty – no models defined yet)
```

Schema: **User** (auth database)

Field	Type	Constraints	Source
<code>_id</code>	ObjectId	Auto-generated by MongoDB	MongoDB
<code>firebaseUid</code>	String	<code>unique: true</code>	Google's Firebase UID
<code>name</code>	String	—	Google profile name
<code>email</code>	String	—	Google email
<code>avatar</code>	String	—	Google profile picture URL
<code>createdAt</code>	Date	<code>Auto (timestamps: true)</code>	Mongoose
<code>updatedAt</code>	Date	<code>Auto (timestamps: true)</code>	Mongoose

Relationships: `User._id` is stored in Redis as `userid` → becomes `x-user-id` header → becomes `Conversation.userId`

Schema: **Conversation** (chat database)

Field	Type	Constraints	Source
<code>_id</code>	ObjectId	Auto-generated	MongoDB

Field	Type	Constraints	Source
<code>title</code>	String	Default: <code>"New Chat"</code>	Set on creation, updated by <code>updateConversation</code>
<code>userId</code>	String	—	From <code>x-user-id</code> header (originally <code>User._id</code>)
<code>createdAt</code>	Date	Auto	Mongoose
<code>updatedAt</code>	Date	Auto	Mongoose

Relationships:

- `userId` → links to `User._id` (but no ref/populate — it's a plain string, not an ObjectId reference)
- `_id` → referenced by `Message.conversationId`

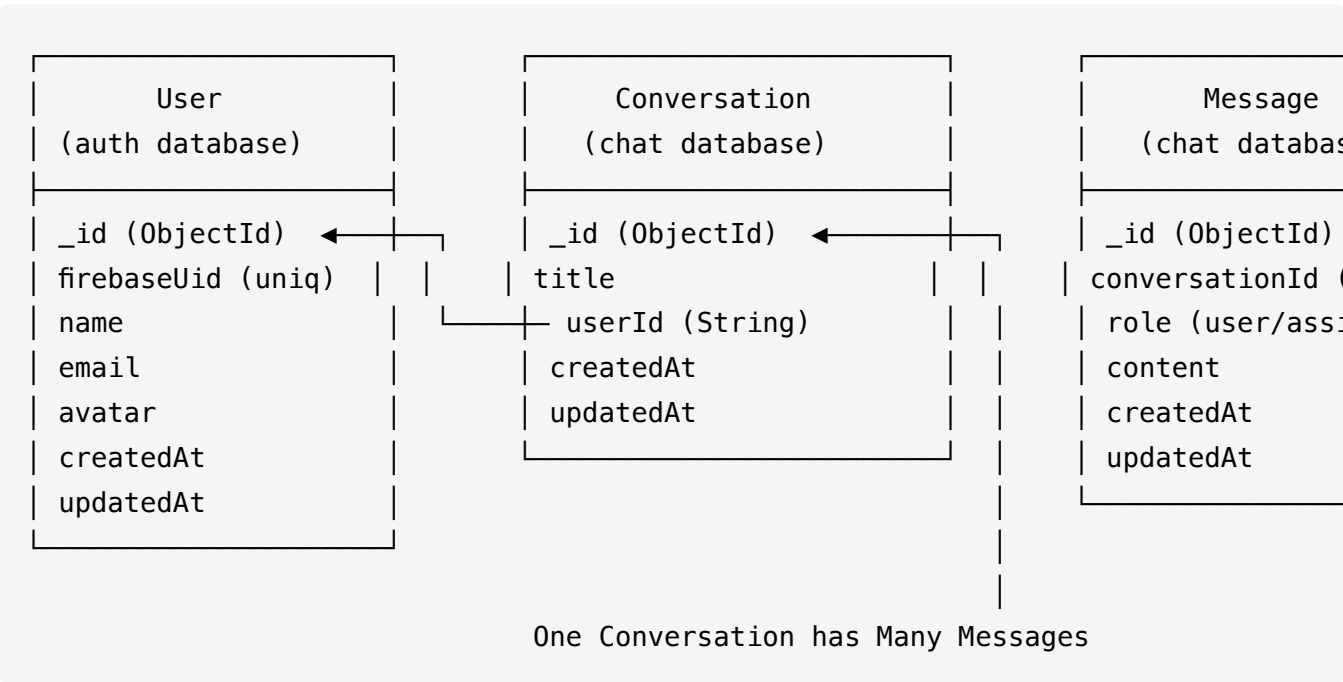
Schema: `Message` (chat database)

Field	Type	Constraints	Source
<code>_id</code>	ObjectId	Auto-generated	MongoDB
<code>conversationId</code>	ObjectId	<code>ref: "Conversation"</code>	Links to parent Conversation
<code>role</code>	String	<code>enum: ["user", "assistant"]</code>	Set during message creation
<code>content</code>	String	—	User's prompt or AI's response
<code>createdAt</code>	Date	Auto	Mongoose
<code>updatedAt</code>	Date	Auto	Mongoose

Relationships:

- `conversationId` → references `Conversation._id`
- `role` → distinguishes who sent the message

Entity Relationship Diagram



⚠ Important: `User._id` (`ObjectId`) is stored as a **String** in `Conversation.userId`, not as an `ObjectId` reference. This means you can't use Mongoose `.populate()` to auto-join them. This is a deliberate choice because `User` lives in a different database (`auth`) than `Conversation` (`chat`).

4. Complete API Endpoints Reference

Gateway Routes (port 8000) — What the browser calls

Method	Path	Auth?	Proxied To	Purpose
POST	/auth/login	✗ No	Auth :8001 /login	Login with Google token
GET	/auth/logout	✗ No	Auth :8001 /logout	Logout (clear session)
GET	/me	✓	Local controller	Get current

Method	Path	Auth?	Proxied To	Purpose
		Yes		user profile
GET	/chat/create-conversation	 Yes	Chat :8002 /create-conversation	Create new conversation
GET	/chat/get-conversation	 Yes	Chat :8002 /get-conversation	List user's conversations
POST	/chat/save-message	 Yes	Chat :8002 /save-message	Save a message
GET	/chat/get-message/:id	 Yes	Chat :8002 /get-message/:id	Get messages for conversation
POST	/chat/update-conversation	 Yes	Chat :8002 /update-conversation	Update conversation title
POST	/agent/chat	 Yes	Agent :8003 /chat	Send prompt to AI agent

Auth Service Internal Routes (port 8001)

Method	Path	Request Body	Response
POST	/login	{ token: "firebase_jwt..." }	{ message: "login successful", user... } + sets cookie
GET	/logout	— (reads cookie)	{ message: "logout successful" } + cookie

Chat Service Internal Routes (port 8002)

Method	Path	Request Body / Params	
GET	/create-conversation	Header: x-user-id	{ _id, t

Method	Path	Request Body / Params	
GET	/get-conversation	Header: x-user-id	[{ _id,
POST	/save-message	{ conversationId, role, content }	{ _id, c
GET	/get-message/:conversationId	URL param: conversationId	[{ _id,
POST	/update-conversation	{ id, title }	{ _id, t

Agent Service Internal Routes (port 8003)

Method	Path	Request Body	Response
POST	/chat	{ prompt, conversationId }	{ aiResponse: "..." }

Internal Server-to-Server Calls (NOT exposed to browser)

Caller	Target	Method	Path	Body
Agent :8003	Chat :8002	POST	/save-message	{ conversationId, role: "user", content: pr
Agent :8003	Chat :8002	POST	/save-message	{ conversationId, role: "assistant", conten

5. How to Start the Entire Project

Startup Order (this order matters!)

Step 1: Start Redis (Docker)

```
cd backend/  
docker-compose up -d
```

Step 2: Start Auth Service (needs Redis + MongoDB)

```
cd backend/services/auth/  
npm run dev          # Starts on port 8001
```

Step 3: Start Chat Service (needs MongoDB)

```
cd backend/services/CHAT/  
npm run dev          # Starts on port 8002
```

Step 4: Start Agent Service (needs MongoDB + Chat Service running)

```
cd backend/services/agent/  
npm run dev          # Starts on port 8003
```

Step 5: Start Gateway (needs all 3 services running)

```
cd backend/gateway/  
npm run dev          # Starts on port 8000
```

Step 6: Start Frontend (needs Gateway running)

```
cd frontend/vite-project/  
npm run dev          # Starts on port 5173
```

Why this order?

- **Redis first:** Auth and Gateway depend on Redis for sessions.
- **Auth before Gateway:** Gateway proxies to Auth — if Auth is down, login breaks.
- **Chat before Agent:** Agent makes HTTP calls to Chat — if Chat is down, Agent crashes.
- **Gateway last (among backends):** It proxies to all services — all must be running.
- **Frontend last:** It calls Gateway — Gateway must be ready.

Expected console output when everything is working:

```
# Auth Service terminal:
auth started at 8001
db connect
redis connected

# Chat Service terminal:
chat started at 8002
db connect

# Agent Service terminal:
agent started at 8003
db connect

# Gateway terminal:
gateway started at 8000
redis connected
```

Common startup errors:

Error	Cause	Fix
<code>ECONNREFUSED 127.0.0.1:6379</code>	Redis not running	Run <code>docker-compose up -d</code> in <code>backend/</code>
<code>ECONNREFUSED 127.0.0.1:8002</code>	Chat service not running	Start Chat service first
<code>MongoServerError: bad auth</code>	Wrong MongoDB credentials	Check <code>.env</code> <code>MONGODB_URL</code>
<code>Cannot find module './config/db.js'</code>	Missing <code>.js</code> extension	ES modules require explicit <code>.js</code>
<code>ERR_MODULE_NOT_FOUND</code>	<code>node_modules</code> not installed	Run <code>npm install</code> in that service folder

6. The `shared/` Folder Pattern

Location: `backend/shared/redis/redis.js`

Why a shared folder?

```
backend/
├── gateway/      ← imports shared/redis/redis.js
├── services/
│   └── auth/     ← imports shared/redis/redis.js
└── shared/
    └── redis/
        └── redis.js ← ONE file, TWO consumers
```

- Both Gateway and Auth Service need the same Redis client.
- Instead of duplicating the Redis connection code in both, it lives in `shared/`.
- Both import it with relative paths:
 - Gateway: `import redis from "../../shared/redis/redis.js"`
 - Auth: `import redis from "../../../shared/redis/redis.js"`

Important: Each import creates a separate connection

Even though both import the same file, they run in **separate Node.js processes** (separate servers). So each gets its own Redis connection. The `shared/` pattern shares **code**, not runtime state.

Why Chat and Agent don't use `shared/redis`:

- Chat Service doesn't need Redis at all (no session checking).
 - Agent Service doesn't need Redis either (no session checking, it receives `x-user-id` via header).
 - Only Gateway (reads sessions) and Auth (writes/deletes sessions) need Redis.
-

7. Docker Compose for Redis

File: `backend/docker-compose.yml`

```
services:
  redis:
    image: redis
    ports:
      - 6379:6379
```

What each line means:

Line	What it does
<code>services:</code>	Top-level key — lists all containers to run
<code>redis:</code>	Name of this service (you pick the name)
<code>image: redis</code>	Use the official <code>redis</code> image from Docker Hub
<code>ports:</code>	Map host ports to container ports
<code>- 6379:6379</code>	<code>hostPort:containerPort</code> — your Mac's 6379 → container's 6379

What is NOT in your docker-compose:

- No `volumes:` — Redis data is lost when container stops (fine for dev sessions)
- No `password` — Redis has no authentication (fine for localhost, dangerous for production)
- No other services — MongoDB is on Atlas (cloud), so no local container needed

Useful commands:

```
docker-compose up -d      # Start in background (-d = detached)
docker-compose down       # Stop and remove container
docker-compose ps         # List running containers
docker-compose logs redis # View Redis logs
docker exec -it <id> redis-cli # Open Redis CLI to inspect data
```

Inspecting Redis data manually:

```
# Inside redis-cli:
KEYS *                # List all keys
GET session-a1b2c3d4-e5f6-... # Read a specific session
TTL session-a1b2c3d4-e5f6-... # Check remaining time to live (seconds)
```

8. The `.env` Files — What Each Service Needs

Gateway `.env`

```
PORT=8000                # Gateway listens here
AUTH_SERVICE=http://localhost:8001 # Proxy target for /auth
CHAT_SERVICE=http://localhost:8002 # Proxy target for /chat
AGENT_SERVICE=http://localhost:8003 # Proxy target for /agent
FRONTEND_URL=http://localhost:5173 # CORS origin whitelist
REDIS_URL=redis://localhost:6379   # Session lookups
```

Auth Service `.env`

```
PORT=8001                # Auth listens here
MONGODB_URL=mongodb+srv://...net/auth # User database
FRONTEND_URL=http://localhost:5173    # Not currently used by Auth
FIREBASE_API_KEY=AIzaSy...           # Not used server-side (Admin SDK uses serviceAccount)
REDIS_URL=redis://localhost:6379      # Session writes/deletes
```

Chat Service .env

```
PORT=8002 # Chat listens here
MONGODB_URL=mongodb+srv://...net/chat # Conversations + Messages database
FRONTEND_URL=http://localhost:5173 # Not currently used
FIREBASE_API_KEY=AIzaSy... # Not used by Chat
REDIS_URL=redis://localhost:6379 # Not currently used by Chat
```

Agent Service .env

```
PORT=8003 # Agent listens here
MONGODB_URL=mongodb+srv://...net/agent # Agent database (currently unused)
GROQ_API_KEY=gsk_... # Groq LLM API key
GOOGLE_API_KEY=AQ.Ab8... # Google Gemini API key
CHAT_SERVICE=http://localhost:8002 # For saving messages via HTTP
FRONTEND_URL=http://localhost:5173 # Not used
FIREBASE_API_KEY=AIzaSy... # Not used
REDIS_URL=redis://localhost:6379 # Not used
```

Frontend .env

```
VITE_FIREBASE_API_KEY=AIzaSy... # Firebase client SDK (public, safe)
VITE_SERVER_URL=http://localhost:8000 # Gateway URL for API calls
```

 **Note:** Many .env files have unused variables (FIREBASE_API_KEY in Chat, REDIS_URL in Agent). These are copy-paste leftovers from creating new services.

9. Package.json Scripts & Dependencies Explained

Why each service has its own `package.json` and `node_modules` :

In a microservices architecture, each service is **independent**. They could be written in different languages. Each has its own:

- `package.json` — declares its own dependencies
- `node_modules/` — installs only what it needs
- `package-lock.json` — locks exact versions

Key dependencies per service:

Dependency	Gateway	Auth	Chat	Agent	Purpose
<code>express</code>	✓	✓	✓	✓	HTTP server framework
<code>dotenv</code>	✓	✓	✓	✓	Load <code>.env</code> files
<code>mongoose</code>	—	✓	✓	✓	MongoDB ODM
<code>cors</code>	✓	—	—	—	Cross-origin requests
<code>cookie-parser</code>	✓	—	—	—	Parse cookies
<code>express-http-proxy</code>	✓	—	—	—	Proxy requests
<code>morgan</code>	✓	—	—	—	HTTP request logging
<code>firebase-admin</code>	—	✓	—	—	Verify Google tokens
<code>firebase</code>	—	✓	—	—	Firestore SDK (unused?)

Dependency	Gateway	Auth	Chat	Agent	Purpose
ioredis	Root	—	—	—	Redis client
@langchain/core	—	—	—	✓	LangChain base
@langchain/groq	—	—	—	✓	Groq LLM provider
@langchain/google-genai	—	—	—	✓	Google Gemini LLM
@langchain/langgraph	—	—	—	✓	Graph workflow engine
axios	—	—	—	✓	HTTP client for server-to-server calls

⚠ **Oddity:** The Gateway `package.json` has `@reduxjs/toolkit` and `react-redux` as dependencies — these are frontend packages and should NOT be in the backend Gateway.

The root `backend/package.json` :

```
{
  "dependencies": {
    "init": "^0.1.2",
    "ioredis": "^5.11.1"
  }
}
```

- `ioredis` is installed at the root level because `shared/redis/redis.js` imports it.
 - Both Gateway and Auth resolve `ioredis` by walking up the directory tree to find it in root `node_modules/`.
-

10. Inter-Service HTTP Calls (Agent → Chat)

The Problem:

When a user sends a prompt, the Agent Service processes it and gets an AI response. But messages need to be saved in the Chat database — which only the Chat Service can access.

The Solution: Internal HTTP calls via axios

```
// agent.controller.js

// Call 1: Save user's message
await axios.post(`${process.env.CHAT_SERVICE}/save-message`, {
  conversationId,
  role: "user",
  content: prompt
})

// Call 2: Save AI's response
await axios.post(`${process.env.CHAT_SERVICE}/save-message`, {
  conversationId,
  role: "assistant",
  content: aiResponse
})
```

Important details:

- **No Gateway in between:** Agent calls Chat directly (`localhost:8002`), NOT through Gateway (`localhost:8000`).
- **No authentication:** These are internal calls — no cookie, no `protect` middleware, no `x-user-id` header.
- **No `x-user-id` header:** The `saveMessage` controller doesn't check `x-user-id` — it only uses `conversationId`, `role`, and `content` from the body.
- **Synchronous flow:** Agent waits for each save to complete before proceeding (`await`).
- **If Chat is down:** The `await axios.post(...)` will throw an error → caught by the try-catch → returns 500 to the frontend.

This pattern is called: Synchronous Request/Response inter-service communication

Other patterns exist (message queues, event-driven) but are more complex. HTTP calls are the simplest form.

11. Error Scenarios & What Breaks

What happens when each component fails:

Component Down	What Breaks	User Experience
Redis	<code>protect</code> middleware can't validate sessions	ALL protected routes return 401. User appears logged out.
Auth Service	Login and logout don't work	Can't sign in. Already-logged-in users still work (sessions in Redis).
Chat Service	Can't create/load conversations or messages	Sidebar shows no conversations. Sending messages fails (Agent can't save).
Agent Service	AI responses fail	User can send messages but gets 500 error. Chat history still works.
MongoDB Atlas	No data persistence	New users can't register. Conversations can't be created.
Gateway	EVERYTHING	Frontend can't reach any backend service.

What happens with an expired session:

```
Browser sends cookie with expired session UUID
→ Gateway protect: redis.get("session-expired-uuid") returns null
→ protect returns: 401 { message: "session expired" }
→ Frontend receives 401
→ ⚠️ Currently: Frontend doesn't handle this – no redirect to login
→ User stays on blank screen until they refresh
```

What happens when router LLM returns unexpected value:

```
User prompt: "asdfghjkl" (gibberish)
→ Router LLM might return: "general" or "unknown" or "chat\n"
→ graph.js conditional edges: switch(state.agent) – no matching case
→ default: break (returns undefined)
→ LangGraph doesn't know which node to go to
→ Graph execution may hang or throw error
```

12. Unanswered "Why" Questions

Why does each service connect to MongoDB on startup even if some don't use it?

The Agent service calls `connectDb()` on startup even though it has no Mongoose models. This is future-proofing — when you implement search/pdf/ppt agents, they might need their own database collections.

Why is the `/auth` route not protected but `/chat` and `/agent` are?

Because `/auth/login` is the endpoint that CREATES the session. The user can't have a session before they log in — it's a chicken-and-egg problem. `/auth/logout` could arguably be protected, but it gracefully handles missing sessions already.

Why does the Agent service save messages instead of the frontend?

1. **Atomicity:** Agent knows both the user prompt and AI response — it saves them together.
2. **Fewer API calls:** Frontend makes 1 call instead of 3 (save user msg → get AI → save AI msg).
3. **Security:** The frontend could be manipulated to send fake "assistant" messages.

Why are there 5 empty agent stubs?

They represent the planned multi-agent architecture:

- **search:** Will use web search APIs to find current information
- **coding:** Will use Gemini (better at code) for code generation
- **pdf:** Will generate PDF documents
- **ppt:** Will generate PowerPoint presentations
- **vision:** Will generate or analyze images

The routing LLM already classifies prompts into these categories. Only the agent logic needs to be implemented.

Why does `search` edge connect to `chat` instead of `_end_`?

```
workflow.addEdge("search", "chat") // search → chat → _end_
```

This is a **pipeline pattern**: The search agent finds information from the web, then the chat agent uses that information to compose a well-formatted response. Search provides raw data, chat provides polished output.

Why is the `Conversation.userId` a String and not an ObjectId reference?

Because `User` lives in the `auth` database and `Conversation` lives in the `chat` database. Mongoose `.populate()` only works within the same database connection. Since these are different microservices with different database connections, a cross-database reference

wouldn't work. Storing it as a plain String is the correct approach for microservices.